Vial Line ML: Training Handoff

Prepared October 28, 2026. Scope: the model-training work only (file-transfer and instance-setup troubleshooting are left out). Headline: no model has been trained yet for the judged final-station task.

1. Status at a glance

Workstream	Status	Notes
Fill-station in_fill classifier	Trained, not usable as-is	ResNet18, best run 68.5% validation accuracy vs a 62.9% always-guess-in_fill baseline. Different station from the judged task. Kept as a training template.
Final-station defect classifier (judged task)	Not trained	No labeled crops yet. Groundwork done: position map, crop generator, camera-stability check.
Alarm and decision logic	Built	20 of 20 scenario tests pass. Thresholds are untuned guesses.
Synthetic data from the digital twin	Not started	Needs Isaac Sim, the station files and scripted defects.

Blockers: (1) no labels exist for the final-station images; (2) the official sample bundle (full images, fixture coordinates, labeled and unlabeled crops, labeling tool) has not been located; (3) the S fixture's 30 positions are unverified.

2. The judged task

Build a system that uses the overhead camera at the final station to check each numbered vial position after each placement. There are two fixtures with 30 fixed positions each: **G** (good) and **S** (scrap). The robot fills the rightmost column top to bottom, then moves one column to the left. Judging: a missed defect costs more than a false alarm.

Class labels describe one target position, not the whole image: `pass`, `missing_cap` (vial present, mouth open; do not confuse the transparent body with an empty hole), `crooked_cap` (tilted or not seated; a leaning vial alone is not enough), `damaged_vial`, and `no_vial` (empty aperture; an error only inside the expected fill pattern).

Context: the line already inspects each vial from the side. That inspection misses some cap defects, and a cap can move, or a robot placement can fail, afterward.

3. Data and assets in hand

- **Real overhead images:** the `cap_seal` folder, 1,181 frames at 640x480 (84 from Sept 22, 1,097 from Sept 23), all unlabeled. The twin README says its `CapSealCam` was solved from the cap_seal clip, so this is very likely the challenge camera. Not yet confirmed with the organizers.
- **Fill-station data (earlier task):** 5 videos; only video 5 is labeled (30 vials, 10 recipes, 845 sampled frames: 509 in fill, 336 between fills).
- **Digital twin:** `csi_line.usda` (a stub that references the station files), `tray_inspection.usdc` (130 MB, 299 prims), camera JSONs for `CapSealCam` and `FillCam`, and two twin renders. The fill_station, magnemover and facility files and the render tools were not seen.
- **Not found:** the official sample bundle named in the brief. The challenge web page could not be fetched.

4. Work completed

4.1 Fill-station classifier (in_fill vs between_fills)

Setup. Script `train_in_fill.py`: pretrained ResNet18, 2 classes, AdamW with learning rate 1e-4, batch size 32, 8 epochs, color jitter. The split is by vial, not by frame, because frames about 2 s apart are near-duplicates. Held-out vials: 3, 9, 15, 21, 27. Idle frames were assigned to the nearest vial by time. That gives 702 train and 143 validation frames (in-fill fraction 0.597 train, 0.629 validation).

Results.

Run	Final val acc	Confusion [[idle correct, idle called fill], [fill called idle, fill correct]]	Notes
Full frame	0.490	[[35, 18], [55, 35]]	Val accuracy bounced between 0.39 and 0.61 while train loss fell from 0.68 to 0.21: overfitting.
Cropped to vial and nozzle	0.685	[[27, 26], [19, 71]]	Fill recall 0.79, precision 0.73; idle recall 0.51. Val accuracy bounced between 0.455 and 0.685, so treat the score as noisy.

The crop is (200, 0, 440, 440) from the 640x480 frame, resized to 192x352. It also removes the burned-in timestamp in the bottom-left corner, a likely label leak. The crop box was confirmed on a video 5 frame.

Findings from the video 5 footage (exploratory color measurements, not trained results):

- Label windows are time-aligned with the video (best agreement at a 0 to +2 s shift; liquid appears 1 to 3 s after each start).
- A thin stream is visible in only about 12% of in-fill frames because dosing is intermittent, and clear T100 liquid is invisible.
- A simple liquid-in-vial color feature reaches AUC 0.79 against in_fill, about what the ResNet achieved. Liquid also stays visible after a fill ends until the vial is swapped, so 25% of idle frames within 4 s after a fill show it. A rising-level feature did not help (AUC 0.54 to 0.67).
- Conclusion: single-frame accuracy has a ceiling well below 100%. Better framings are predicting the finished vial's contents (total_ml or recipe), smoothing over time, and cross-validating by vial.

Status. Superseded by the final-station task. The training loop, vial-grouped split and evaluation printout are reusable.

4.2 Final-station groundwork

- **Camera model.** `CapSealCam` is overhead (about 2.37 m up, axis nearly straight down), 640x480, focal length 652.91 px, lens k1 = -0.1527. Projecting twin geometry through this model lands on the correct vials in the twin render.
- **Position map.** In the twin, G01 to G30 are numbered column by column: rightmost column first, top to bottom, then leftward (5 rows by 6 columns). Pitch is about 30.5 px across and 36.2 px down, and a cap is about 26 px wide, so defects are small. Saved as `twin_positions_px.csv` (pixel center of each position).
- **S fixture.** The twin has only S01 to S05, tilted along the plate edge, with caps on S01 and S04 only. These are not aperture centers, and the other S positions are unknown.
- **Check on real frames.** An overlay on a Sept 22 frame (17:15:25) put the circles on the apertures. Caps were at positions 1 to 5 and 8, a washed-out cap at 6 (glare, which a color rule misses), a possible open vial at 7 (unverified), and positions 9 to 30 empty. Registering 92 Sept 23 frames against that frame gave a shift of about 0.1 px, so one positions file serves all 1,181 frames (G only). Visual overlays on two Sept 23 frames agree.
- **Crop generator `crops.py`.** One tight (72 px) and one context (144 px) crop per position, centered on the target, plus an `index.csv` that maps each crop to its image, fixture and position, with a blank `label` column. It accepts a folder and skips macOS `._` files. 60 frames produced 1,800 crop pairs in about 4 s (102 MB); the full set is about 35,000 pairs (about 2 GB as PNG).
- **Decision logic `decision.py`.** Turns per-position predictions into OK, REVIEW or HOLD. For G, any defect, uncertain vial (P(pass) below 0.85), empty position inside the fill pattern, or vial that changed after placement means HOLD; positions beyond the placement count may be empty. For S, defects are expected, a good-looking vial is logged as a possible false reject, and an empty position inside the pattern is REVIEW. Defaults: defect flag at P(defect) 0.20 (0.40 beyond the fill pattern) and empty at P(no_vial) 0.50. It includes `pick_threshold()` to choose a threshold for a target defect recall. `test_decision.py` holds 20 made-up scenarios, all passing.
- **`summarize_twin.py`.** Extracts a prim summary from a USD file so the geometry can be inspected without moving huge files. The tray station has named G01 to G30 tubes and caps and S01 to S05 tubes. The twin README says only the fill station carries `synthetic_class` labels, so tray-station defect labels would have to be authored.

5. Reproducing on the instance (training-relevant notes)

- A Python 3.12 virtual environment is required because system pip is blocked. Packages: `torch torchvision pandas scikit-learn pillow opencv-python-headless`.
- cuDNN conflict: `LD_LIBRARY_PATH` points at the system CUDA 13.2, which clashes with the libraries bundled in the venv's PyTorch. Run with `env -u LD_LIBRARY_PATH python train_in_fill.py --data <dataset> --epochs 8`. Fallback: set `torch.backends.cudnn.enabled = False`.
- The copy of `train_in_fill.py` on the instance was edited by hand and patched with the crop. The archived copy in the outputs folder is the original without the crop. On the instance the crop script is named `crop.py`.
- To crop a folder of frames: `python crop.py <images folder> --positions twin_positions_px.csv --fixture G --out crops_all/`.

6. Assumptions and unverified items

- Numbering follows fill order for G (verified in the twin and consistent with the Sept 22 frame). S numbering is assumed to be the same.
- `cap_seal` is the challenge camera (likely, not confirmed).
- The thresholds in `decision.py` are guesses and must be tuned on real validation data.
- The twin render is cleaner than real frames, so expect a domain gap, and evaluate on real images only.
- Without a placement count or an order-complete signal, a missed last placement cannot be detected.
- The fill-station findings come from crude color features on one video, not from a controlled study.

7. Recommended next steps

1. Locate the official sample bundle and labeling tool, confirm the camera, and get the S fixture coordinates (or detect the S holes).
2. Build pre-labels for the real crops: fill order for empty positions, cap color for likely capped, and an ambiguous queue for human review. Count classes after review.
3. Train the position classifier on tight plus context crops (pretrained backbone, class weights, brightness and contrast augmentation for glare). Split by source frame or order, never by crop. Report per-class recall, a confusion matrix and the fraction of

orders where any defect slips through.

4. Tune the defect threshold with `pick_threshold()` (for example a 98% recall target) and report the false-alarm cost.
5. Use the twin only for rare cases (crooked caps, missing caps, overlapping bodies). It needs Isaac Sim, the station files and scripted defects.
6. Run an end-to-end test of the classifier plus `decision.py` on frame sequences with placement counts.

8. File inventory

File	Purpose
`vial_inspection/crops.py`	Per-position crop generator (tight and context)
`vial_inspection/decision.py`	Alarm logic and threshold picker
`vial_inspection/test_decision.py`	20 scenario tests
`vial_inspection/twin_positions_px.csv`	Pixel centers of G positions (S rows are not aperture centers)
`vial_inspection/summarize_twin.py`	USD prim summarizer
`vial_fill_archive/train_in_fill.py`	Fill-station training script, original without the crop; reusable template